SECTION 1 — COURSE INFORMATION

Institution	Golden Success College Inc.
College / Department	IT Department
Course Title	Data Structures and Algorithm
Course Code	CC104
Credit Units	3 Units (2 hrs Lecture + 3 hrs Laboratory per week)
Course Type	Lecture and Laboratory
Lecture Hours/Week	2 hours
Laboratory Hours/Week	3 hours
Pre-requisite(s)	Mathematics and Programming (or equivalent introductory programming course)
Year Level & Semester	2nd Year, 1st Semester
Academic Year	2026-2027
Prepared by	Jake Lorenze B. Sabal
Approved by	Nicanor IV B. Nuñez

SECTION 2 — INSTITUTIONAL CONTEXT

1. Vision, Mission, Goals, and Objectives (VMGO)

VISION

A premier learning institution in the Philippines dedicated to holistic and world-class standard of education.

Course Contribution: Data Structures and Algorithm develops rigorous computational thinking and problem-solving skills using industry-relevant tools, directly supporting the institution's vision of producing world-class, technically excellent IT graduates.

MISSION

This institution is committed to the upliftment of the country by making available to the public high quality and internationally competitive education in various fields of sciences, values formation and trade skills for effective development of innovators, future leaders, entrepreneurs and workforce in the country.

Course Contribution: By mastering data structures and algorithm analysis, students acquire high-demand computing skills essential for innovation and leadership in the IT industry, directly advancing the institution's mission of developing a competitive Filipino workforce.

GOAL

To train and develop students to become INNOVATOR, ENTREPRENEUR and FUTURE LEADER of the Philippines.

OBJECTIVES

1. Acquire the greatest possible understanding of himself and appreciation of his worthiness as a member of society at large (Values: SELF-ESTEEM);
2. Acquire understanding and appreciation of persons belonging to other social and cultural groups. (Values: UNDERSTANDING OTHERS);

3. Acquire to the fullest possible extent, the mastery of the basic skills in the use of words, numbers, scientific skills and critical thinking skills. (Values: INTEREST IN THE SCHOOL);
4. Develop a positive attitude toward the learning process. (Values: INTEREST IN THE SCHOOL);
5. Acquire the habits and attitudes associated with responsible citizenship and leadership in the Community. (Values: RESPONSIBLE CITIZENSHIP);
6. Develop the Christian values especially love, justice and peace. (Values: CHRISTIAN VALUES);
7. Acquire good habits and understanding of conditions necessary for maintaining physical and emotional wellbeing. (Values: HEALTH HABITS);
8. Take the opportunity and encouragement to be creative in one or more fields of learning. (Values: CREATIVITY);
9. Understand the opportunity to prepare for a productive life and help each to take full advantage of these opportunities. (Values: CAREER AWARENESS); and
10. To help each child to realize that to live a fully human life is to live a life worthy of the image and likeness of God. (Values: REVERENCE TO GOD).

2. Program Educational Objectives (PEOs)

Within 3–5 years after graduation, graduates of the BSIT program are expected to:

PEO1. Demonstrate professional competence in applying computing and IT knowledge to design and deliver effective technology solutions in industry or private practice.
PEO2. Exhibit continuous professional growth through lifelong learning, industry certifications, and active engagement with the IT professional community.
PEO3. Lead or contribute effectively to multidisciplinary teams, applying sound ethical judgment and social responsibility in technology-driven environments.
PEO4. Initiate and manage IT projects or technology-based enterprises that address organizational and societal needs.
PEO5. Communicate technical information clearly and effectively to both technical and non-technical stakeholders in professional and community settings.

SECTION 3 – COURSE DESCRIPTION

Data Structures and Algorithm covers the design, analysis, and implementation of fundamental data structures and algorithms for solving computational problems using an object-oriented programming language (C#). The course begins with a review and extension of object-oriented programming principles – including abstraction, encapsulation, inheritance, polymorphism, and exception handling – before progressing to core data structures: arrays, lists, linked lists, stacks, queues, trees, and binary search trees. Students will study the algorithms that operate on these structures and apply them to practical programming problems. The course further develops competency in algorithm analysis, introducing asymptotic complexity notation (Big-O) to enable systematic evaluation and comparison of algorithm efficiency. Advanced searching techniques (sequential, binary, hashing) and key sorting algorithms are also covered. This course is a cornerstone of the BSIT curriculum, providing the theoretical and practical foundation required for software development, systems programming, database management, and all upper-level computing courses.

SECTION 4 – PROGRAM OUTCOMES (POs)

The following Program Outcomes are defined under CHED CMO No. 25, series of 2015 (BSIT PSG). Graduates of the BSIT program should be able to:

PO1. Apply knowledge of computing, science, and mathematics appropriate to the discipline.
PO2. Understand best practices and standards and their applications.
PO3. Analyze complex problems and identify and define the computing requirements appropriate to their solution.
PO4. Identify and analyze user needs and take them into account in the selection, creation, evaluation, and administration of computer-based systems.
PO5. Design, implement, and evaluate a computer-based system, process, component, or program to meet desired needs and requirements under various constraints.
PO6. Integrate IT-based solutions effectively in a given environment.
PO7. Apply knowledge through the use of current techniques, skills, tools, and practices necessary for the IT profession.
PO8. Function effectively as a member or leader of a development team, recognizing the different roles within a team to accomplish a common goal.
PO9. Assist in the creation of an effective IT project plan.
PO10. Communicate effectively with the computing community and with society at large about complex computing activities through logical writing, presentations, and clear instructions.
PO11. Analyze the local and global impact of computing information technology on individuals, organizations, and society.
PO12. Understand professional, ethical, legal, security, and social issues and responsibilities in the utilization of information technology.
PO13. Recognize the need for and engage in planning self-learning and improving performance as a foundation for continuing professional development.

SECTION 5 — COURSE OUTCOMES (COs)

Upon successful completion of this course, students should be able to:

CO1. Apply object-oriented programming principles — including abstraction, encapsulation, inheritance, polymorphism, generics, and exception handling — using C# to construct well-structured programs.
CO2. Implement and demonstrate the operations of fundamental data structures (arrays, linked lists, stacks, queues, trees, and binary search trees) using C#.
CO3. Design and develop C# programs that select and apply appropriate data structures and algorithms to solve given computational problems.
CO4. Analyze the time complexity of algorithms using asymptotic notation (Big-O) and evaluate and compare the efficiency of different algorithms and data structure operations.

SECTION 6 — CO–PO MAPPING MATRIX

Legend: I = Introduced | D = Developed | E = Enhanced/Demonstrated

Course Outcome	PO1	PO2	PO3	PO5	PO7	PO10	PO13
CO1	I	I		I	D		I
CO2	D	D	I	D	D		
CO3	D	D	D	E	E	I	
CO4	E	D	E	D			D

SECTION 7 — CONSTRUCTIVE ALIGNMENT TABLE

Course Outcome	Teaching-Learning Activities (TLA)	Assessment Tasks (AT)
CO1. Apply OOP principles using C# to construct well-structured programs.	Lecture/Discussion; Demonstration; Laboratory Exercise; Problem-Based Learning	Laboratory Output; Quiz; Seatwork; Written Exam (Prelim)
CO2. Implement and demonstrate operations of fundamental data structures using C#.	Lecture/Discussion; Demonstration; Laboratory Exercise; Problem-Based Learning; Case Study	Laboratory Output; Seatwork; Quiz; Written Exam (Midterm/Semi-Final); Data Structures Portfolio (Output 1)
CO3. Design and develop C# programs applying appropriate data structures and algorithms.	Problem-Based Learning; Project-Based Learning; Laboratory Exercise; Collaborative Learning	Laboratory Output; Algorithm Design Project (Output 2); Oral Presentation; Written Exam (Final)
CO4. Analyze time complexity using Big-O and compare algorithm efficiency.	Lecture/Discussion; Case Study; Demonstration; Problem-Based Learning; Collaborative Learning	Seatwork (complexity analysis); Quiz; Written Exam (Semi-Final/Final); Algorithm Analysis Report

SECTION 8 — MAJOR COURSE OUTPUTS

The following major outputs serve as evidence of Course Outcome attainment. Rubrics for all outputs are described in Section 13.

Output No.	CO Addressed	Description of Output	Due (Week/Period)
Output 1	CO1, CO2	Data Structures Implementation Portfolio: A compiled set of working C# programs demonstrating the implementation and operations of at least four (4) different data structures studied in the course (e.g., linked list, stack, queue, BST). Each program must include: (a) correct implementation of the data structure and its core operations, (b) a sample run demonstrating functionality, (c) brief inline documentation. Submitted as a zipped project folder with a cover page.	Week 10 (Midterm Period)
Output 2	CO3, CO4	Algorithm Design and Analysis Project: A complete, documented C# program that solves a defined real-world or academic computational problem using at least two (2) data structures and two (2) algorithms (including at least one searching and one sorting algorithm).	Week 18 (Final Period)

		Submission includes: (a) problem statement and program documentation, (b) source code with inline comments, (c) Big-O complexity analysis for each algorithm used, (d) comparison of algorithm alternatives considered, and (e) sample program output. Students present and defend their work during the Final period.	

SECTION 9 — LEARNING PLAN (WEEKLY COURSE SCHEDULE)

Week	Intended Learning Outcomes (ILO)	Topics & Content	Teaching-Learning Activities (TLA)	Assessment Tasks (AT)
1	After this week, students should be able to: (1) State course expectations, policies, and requirements; (2) Explain data abstraction and the concept of Abstract Data Types (ADTs); (3) Review C# class syntax including fields, properties, methods, and constructors. [CO1]	Course Orientation & Policies Data, Data Types, and Abstraction Abstract Data Types (ADTs): Concept and Importance Review of C# Class Syntax: Fields, Properties, Methods, Constructors Access Modifiers: public, private, protected	Lecture/Discussion; Demonstration; Reading Assignment	Seatwork 1: C# Class Design Exercise; Oral Recitation
2	After this week, students should be able to: (1) Apply encapsulation using C# properties and access modifiers; (2) Implement inheritance to create class hierarchies in C#; (3) Demonstrate polymorphism using method overriding and virtual/override keywords. [CO1]	Encapsulation: Properties, Getters, Setters Inheritance: Base and Derived Classes Method Overriding: virtual, override, abstract Polymorphism: Compile-time vs. Runtime The object Base Class	Lecture/Discussion; Demonstration; Laboratory Exercise	Lab Activity 1: OOP Hierarchy Program; Quiz 1
3	After this week, students should	Operator Overloading in C#	Lecture/Discussion; Demonstration;	Seatwork 2: Exception

	be able to: (1) Implement operator overloading in C# classes; (2) Write exception handling code using try-catch-finally blocks; (3) Apply memory management concepts including the use of the C# Garbage Collector. [CO1]	Exception Handling: try, catch, finally, throw Common Exception Types: IndexOutOfRangeException, NullReferenceException Memory Management: Stack vs. Heap, Garbage Collection using Statement and IDisposable Interface	Problem-Based Learning	Handling Programs; Quiz 2
4	After this week, students should be able to: (1) Define and use generic classes and methods in C# to write type-safe, reusable code; (2) Apply generic constraints to restrict type parameters; (3) Explain the advantages of generics over non-generic alternatives. [CO1]	Introduction to Generics in C# (equivalent to C++ Templates) Generic Classes: class Stack<T>, class Queue<T> Generic Methods Generic Constraints (where T : class, where T : IComparable) Built-in Generic Collections: List<T>, Dictionary<K,V>	Lecture/Discussion; Demonstration; Laboratory Exercise	Lab Activity 2: Generic Data Container; Seatwork 3
5	–	PRELIM EXAMINATION (Coverage: Weeks 1–4) Review: OOP Principles, C# Classes, Generics, Exception Handling	Review Discussion; Q&A	Summative Written Exam (Prelim)
6	After this week, students should be able to: (1) Declare, initialize, and manipulate static and dynamic arrays in C#; (2) Perform common	Arrays in C#: Declaration and Initialization Static vs. Dynamic Arrays Array Traversal: Indexing and foreach Loops	Lecture/Discussion; Demonstration; Laboratory Exercise	Lab Activity 3: Array Manipulation Programs; Quiz 3

	array operations: traversal, insertion, deletion, and search; (3) Compare one-dimensional and multi-dimensional arrays in terms of structure and access. [CO2]	Array Operations: Insert, Delete, Search Multi-Dimensional Arrays and Jagged Arrays Built-in Array Methods: Sort(), Copy(), Resize()		
7	After this week, students should be able to: (1) Describe the List ADT and its behavioral contract; (2) Implement a List using arrays (static and dynamic resizing); (3) Perform and trace basic List operations: add, remove, get, set, contains, size. [CO2]	The List ADT: Definition and Interface Array-Based List Implementation Dynamic Array Resizing Strategy List Operations: add(), remove(), get(), set(), contains(), size() Comparison: Built-in List<T> vs. Custom Implementation Time Complexity of List Operations	Lecture/Discussion; Demonstration; Problem-Based Learning	Seatwork 4: List ADT Implementation; Quiz 4
8	After this week, students should be able to: (1) Explain the structure of singly and doubly linked lists using node and pointer concepts; (2) Implement a singly linked list in C# with add, remove, search, and traverse operations; (3) Trace linked list operations step-by-step using diagrams. [CO2]	Introduction to Linked Lists: Nodes and References Singly Linked List: Structure and Implementation in C# Linked List Operations: AddFirst, AddLast, Remove, Search, Traverse Doubly Linked List: Structure and Bidirectional Traversal Comparison: Array-Based List vs. Linked List Circular Linked List: Concept and Use Cases	Lecture/Discussion; Demonstration; Laboratory Exercise	Lab Activity 4: Linked List Implementation; Seatwork 5

9	After this week, students should be able to: (1) Explain the LIFO behavior of stacks and FIFO behavior of queues; (2) Implement array-based and linked-list-based stacks and queues in C#; (3) Apply stacks to solve problems such as expression evaluation and bracket matching. [CO2]	Stack ADT: LIFO Behavior and Interface Stack Implementation: Array-Based and Linked-List-Based Stack Applications: Bracket Matching, Expression Evaluation (Infix/Postfix) Queue ADT: FIFO Behavior and Interface Queue Implementation: Array-Based and Linked-List-Based Queue Applications: Print Spooler, Task Scheduling Deque (Double-Ended Queue): Concept	Lecture/Discussion; Demonstration; Problem-Based Learning; Laboratory Exercise	Lab Activity 5: Stack & Queue Programs; Quiz 5
10	—	MIDTERM EXAMINATION (Coverage: Weeks 6–9) Review: Arrays, List ADT, Linked Lists, Stacks, Queues	Review Discussion; Q&A	Summative Written Exam (Midterm); Output 1 Submission (Data Structures Portfolio – CO1, CO2)
11	After this week, students should be able to: (1) Describe the structure and properties of binary trees using tree terminology (root, leaf, height, depth, level); (2) Implement a binary tree node class in C#; (3) Perform and trace pre-order, in-order, and post-order tree traversals	Tree Data Structure: Terminology and Properties Binary Trees: Structure and Types (Full, Complete, Perfect) Binary Tree Node Implementation in C# Tree Traversals: Pre-order, In-order, Post-order (Recursive) Level-Order Traversal (BFS using a Queue)	Lecture/Discussion; Demonstration; Collaborative Learning; Laboratory Exercise	Lab Activity 6: Binary Tree Traversal; Seatwork 6

	recursively. [CO2]	Applications of Binary Trees: Expression Trees		
12	After this week, students should be able to: (1) Explain the Binary Search Tree (BST) property and its advantage for efficient searching; (2) Implement BST insert, search, and delete operations in C# recursively; (3) Construct and trace a BST given a sequence of insertions and deletions. [CO2]	Binary Search Tree (BST): BST Property BST Operations: Search (Find), Insert, Delete Recursive BST Implementation in C# BST Traversal for Sorted Output (In-order) General (Non-Binary) Trees: N-ary Tree Representation BST Degeneracy and Balanced Trees: Concept Overview	Lecture/Discussion; Demonstration; Problem-Based Learning; Laboratory Exercise	Lab Activity 7: BST Implementation; Quiz 6
13	After this week, students should be able to: (1) Define algorithm analysis and explain why efficiency matters; (2) Express the time complexity of algorithms using Big-O, Big-Omega, and Big-Theta notation; (3) Analyze and compare the time complexity of iterative and recursive algorithms. [CO4]	Algorithm Analysis: What and Why Time Complexity: Best, Average, Worst Case Asymptotic Notation: Big-O, Big-Omega (Omega), Big-Theta Common Complexity Classes: O(1), O(log n), O(n), O(n log n), O(n^2) Analyzing Iterative Code: Loops and Nested Loops Analyzing Recursive Code: Recurrence Relations (Basic) Space Complexity	Lecture/Discussion; Case Study; Problem-Based Learning	Seatwork 7: Complexity Analysis Exercises; Quiz 7
14	—	SEMI-FINAL EXAMINATION (Coverage: Weeks 11–13) Review: Binary Trees, BST,	Review Discussion; Q&A	Summative Written Exam (Semi-Final)

		Algorithm Analysis (Big-O)		
15	After this week, students should be able to: (1) Implement and trace sequential search and binary search algorithms in C#; (2) Analyze the time complexity of each search algorithm (O(n) vs. O(log n)); (3) Compare the conditions under which binary search can be applied. [CO3, CO4]	Sequential (Linear) Search: Algorithm and C# Implementation Binary Search: Algorithm, Pre-condition (sorted array), and C# Implementation Comparison: Sequential vs. Binary Search – Time Complexity Ordered Lists and Search Optimization Recursive Binary Search Implementation Built-in Search Methods in C#: Array.BinarySearch(), List.Contains()	Lecture/Discussion; Demonstration; Problem-Based Learning; Laboratory Exercise	Lab Activity 8: Search Algorithms Comparison; Seatwork 8
16	After this week, students should be able to: (1) Explain the concept of hashing and the role of hash functions; (2) Implement a basic hash table in C# using open addressing and chaining for collision resolution; (3) Evaluate the time complexity of hash table operations under ideal and worst-case conditions. [CO3, CO4]	Introduction to Hashing: Concept and Motivation Hash Functions: Design Criteria and Common Examples Hash Tables: Structure and Direct Addressing Collision Resolution: Open Addressing (Linear Probing, Quadratic Probing) Collision Resolution: Separate Chaining Load Factor and Rehashing C# Dictionary<K,V> as Hash Table Time Complexity: O(1) average, O(n) worst-case	Lecture/Discussion; Demonstration; Laboratory Exercise; Case Study	Lab Activity 9: Hash Table Implementation; Quiz 8

17	After this week, students should be able to: (1) Implement and trace at least four (4) sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort; (2) Analyze the time complexity of each sorting algorithm; (3) Select an appropriate sorting algorithm based on problem constraints. [CO3, CO4]	Sorting Algorithms Overview O(n^2) Algorithms: Bubble Sort, Selection Sort, Insertion Sort – Implementation & Trace O(n log n) Algorithms: Merge Sort (Divide & Conquer), Quick Sort (Partitioning) Comparison of Sorting Algorithms: Time and Space Complexity In-place vs. Stable Sorting Built-in Sorting: Array.Sort(), List.Sort() in C# Output 2 Workshop: Project Progress Check & Code Review	Lecture/Discussion; Demonstration; Problem-Based Learning; Project-Based Learning	Lab Activity 10: Sorting Algorithms Implementation; Assignment: Output 2 Draft
18	–	FINAL EXAMINATION (Coverage: Weeks 15–17 + Cumulative) Output 2 Presentation and Defense	Program Presentation and Defense	Summative Written Exam (Final); Output 2 Submission and Oral Defense (CO3, CO4)

SECTION 10 – GRADING SYSTEM

The grading system is applied uniformly across all four examination periods (Prelim, Midterm, Semi-Final, Final).

Component	Weight per Period	CO Addressed
Major Examination (Written)	30%	CO1, CO2, CO3, CO4
Major Course Output (Program)	45%	CO1, CO2, CO3, CO4
Quizzes	15%	CO1, CO2, CO4
Class Participation	5%	All COs
Attendance	5%	All COs
TOTAL	100%	

Period Grade Computation:
Period Grade = (Major Exam x 0.30) + (Course Output x 0.45) + (Quizzes x 0.15) + (Participation x 0.05) + (Attendance x 0.05)

Running Average Computation:
After Prelim Period : Running Grade = Prelim Grade
After Midterm Period : Running Grade = (2/3 x Prelim) + (1/3 x Midterm)
After Semi-Final : Running Grade = (2/3 x Midterm) + (1/3 x Semi-Final)
After Final Period : Running Grade = (2/3 x Semi-Final) + (1/3 x Final Grade)

Passing Grade: A student must accumulate a final running grade of at least 60% (or equivalent passing mark per institutional policy) to pass the course.

SECTION 11 — COURSE POLICIES

1. ATTENDANCE

Students are expected to attend all scheduled lecture and laboratory sessions. In accordance with CHED policy, the maximum allowable absences is 20% of total class hours per semester. Exceeding this limit shall result in a grade of DROPPED (DRP) regardless of academic standing. Three (3) instances of tardiness (arriving more than 15 minutes after the scheduled start) shall be counted as one (1) absence.

2. ACADEMIC INTEGRITY

All programs, written outputs, and examination answers must be the student's own original work. Code plagiarism — including copying another student's program, submitting code from online repositories without attribution, or using AI-generated code as one's own without declaration — is strictly prohibited and will result in a grade of zero (0) for the affected output. Repeat offenses may be subject to disciplinary action in accordance with institutional policy.

3. LATE SUBMISSIONS

Major course outputs submitted after the stated deadline shall incur a 10% deduction per calendar day late, for a maximum of three (3) days. Outputs submitted more than three (3) days after the deadline shall receive a grade of zero (0) unless a written request for extension was approved by the instructor prior to the due date.

4. LABORATORY CONDUCT

During laboratory sessions, all computers must be used strictly for course-related tasks. Students are responsible for saving and backing up their own work; the instructor assumes no responsibility for unsaved or lost files. Food and beverages are not permitted in the laboratory. Students must properly shut down their workstations and return equipment to its original state at the end of each session.

SECTION 12 – REFERENCES AND LEARNING RESOURCES

Primary Textbook

Shaffer, C. A. (2013). Data structures and algorithm analysis in Java (3rd ed.). Dover Publications. ISBN 978-0-486-48581-2.

Supplementary Books

Malik, D. S. (2013). Data structures using Java. Cengage Learning. ISBN 978-1-449-62807-9.
Weiss, M. A. (2011). Data structures and algorithm analysis in Java (3rd ed.). Addison-Wesley.
Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.
Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data structures and algorithms in Java (6th ed.). Wiley.

Online Resources / Websites

Microsoft. (n.d.). C# documentation. Retrieved from https://docs.microsoft.com/en-us/dotnet/csharp/
GeeksforGeeks. (n.d.). Data structures tutorial. Retrieved from https://www.geeksforgeeks.org/data-structures/
Visualgo. (n.d.). Visualgo: Visualising data structures and algorithms through animation. Retrieved from https://visualgo.net
Big-O Cheat Sheet. (n.d.). Know thy complexities. Retrieved from https://www.bigocheatsheet.com

Development Tools

Microsoft. (n.d.). Visual Studio Community [IDE for C# development]. Retrieved from https://visualstudio.microsoft.com/vs/community/
Microsoft. (n.d.). Visual Studio Code [Source code editor]. Retrieved from https://code.visualstudio.com
Microsoft. (n.d.). .NET SDK. Retrieved from https://dotnet.microsoft.com/en-us/download

SECTION 13 – RUBRICS FOR MAJOR COURSE OUTPUTS

Rubric 1: Data Structures Implementation Portfolio (Output 1) – CO1, CO2

Criterion	Outstanding (4)	Proficient (3)	Developing (2)	Beginning (1)	Weight
Correctness of OOP Design (CO1)	All programs correctly apply OOP principles (encapsulation, inheritance, generics, exception handling); classes are well-designed and cohesive.	Most programs correctly apply OOP principles; minor design inconsistencies.	Some programs apply OOP partially; notable design issues in class structure.	OOP principles are largely absent or incorrectly applied.	25%
Correctness of Data	All required data	Most operations	Some operations	Implementations are	40%

Structure Implementation (CO2)	structure operations are correctly implemented and produce accurate output for all test cases.	are correctly implemented; one or two minor functional errors.	are correctly implemented; notable functional errors in at least one data structure.	largely incorrect or incomplete; most operations do not function.	
Code Quality & Documentation	Code is well-structured, consistently formatted, uses meaningful identifiers, and includes complete comments for all classes and key methods.	Code is mostly well-structured with adequate comments; minor style issues.	Code has inconsistent formatting or sparse comments; readability is reduced.	Code is poorly structured, hard to read, and has no meaningful comments.	20%
Completeness of Portfolio	Portfolio contains all required data structures (minimum 4), each with a cover page, source code, and sample output.	Portfolio contains most required items; one element is missing.	Portfolio is incomplete; two or more elements are missing.	Portfolio is largely incomplete; fewer than half of the required items are present.	15%

Rubric 2: Algorithm Design and Analysis Project (Output 2) – CO3, CO4

Criterion	Outstanding (4)	Proficient (3)	Developing (2)	Beginning (1)	Weight
Problem-Solving & Data Structure Selection (CO3)	Problem is clearly defined; the chosen data structures are optimal for the problem and their selection is well-justified.	Problem is adequately defined; data structure choices are appropriate with brief justification.	Problem definition is partial; data structure choices are acceptable but not well-justified.	Problem definition is vague; data structure choices are arbitrary or inappropriate.	25%
Program Correctness & Functionality (CO3)	Program is fully functional, solves the stated	Program is mostly functional; minor logic errors that	Program is partially functional; core functionality	Program does not function correctly; significant logic or	30%

	problem completely and correctly using the required structures and algorithms.	do not prevent core functionality.	works but some features are incomplete.	runtime errors.	
Algorithm Complexity Analysis (CO4)	Big-O analysis for all algorithms is correct, clearly explained, and includes comparison of at least two alternative approaches with justification.	Big-O analysis is mostly correct; comparison is present but lacks depth.	Big-O analysis is partially correct or present for only some algorithms.	Big-O analysis is absent or largely incorrect.	25%
Presentation & Defense (CO3, CO4)	Student presents program logic and complexity analysis clearly and confidently; answers all questions accurately and insightfully.	Student presents adequately; explains most content; answers most questions.	Student presents with difficulty; partial explanations; struggles to answer questions.	Student cannot present or defend the work effectively.	20%

Legal Basis: CMO 46 s. 2012 | CMO 25 s. 2015 (BSIT PSG) | PQF RA 10968 (Level 6)